

ملخص مشروع Startup Validator AI

وثيقة تعريفية مختصرة قابلة للتطوير

الغرض من هذه الوثيقة هو توضيح فكرة المشروع، نطاق النسخة الحالية، طريقة عمل النظام، المكونات الأساسية، والقرارات التقنية المعتمدة بشكل منظم يمكن مشاركته مع فريق تقني أو دكتور الجامعة أو إضافته إلى مستودع المشروع.

التوصيف الصحيح للمشروع

النظام ليس Chatbot عاماً لتقديم رأي عشوائي حول فكرة Startup، بل منصة تقييم أولي منظمة تعتمد على عدة Agents متخصصة. كل Agent يحلل الفكرة من زاوية مختلفة، ثم يحسب النظام الدرجة النهائية برمجياً ويولد تقريراً واضحاً للمستخدم.

القرار المعتمد	البند
تطبيق ويب لتحليل أفكار الشركات الناشئة وعرض التقارير.	نوع المنتج
تحليل الجانب التجاري والسوق والمخاطر وإنتاج تقرير تقييم منظم.	الهدف الأساسي
Hugging عبر Qwen باستخدام Multi-Agent System Face API.	الذكاء الاصطناعي
Agents وخدمات منفصلة للـ Django مع ORM والـ Validation.	الباكند
PostgreSQL.	قاعدة البيانات
Logout مع Refresh Token و JWT Access Token Revocation.	المصادقة
LLM وليس بواسطة الـ Python داخل ScoreCalculator.	حساب النتيجة
يعملون بالتوازي باستخدام Risk وMarket و Business Thread Pool Executor.	تنفيذ الـ Agents
حفظ جميع التحليلات والتقرير النهائي لكل مستخدم.	التقارير

0. فهرس مختصر

- الفكرة العامة والهدف من المشروع
- الفكرة المعتمدة ونطاق النسخة الحالية
- الأطراف والمكونات الرئيسية داخل النظام
- طريقة عمل الـ Multi-Agent System
- الميزات المعتمدة والميزات المؤجلة
- التقنيات المستخدمة
- مكونات النظام وقاعدة البيانات

وثيقة ملخص مشروع - Startup Validator AI

- مسارات الاستخدام
- نظام التقييم والدرجات
- حالة التنفيذ والخطوات التالية
- المخاطر والضوابط الأساسية

1. الفكرة العامة والهدف من المشروع

المشروع عبارة عن منصة تساعد المستخدم على تقييم فكرة شركة ناشئة قبل البدء بنائها. يدخل المستخدم إلى النظام، يسجل الدخول، ثم يكتب عنوان الفكرة ووصفها. بعد ذلك يمرر النظام نفس الفكرة إلى عدة وكلاء ذكاء اصطناعي متخصصين. يحلل Business Agent الجانب التجاري، ويحلل Market Agent السوق والطلب والمنافسة، بينما يركز Risk Agent على المخاطر. بعد اكتمال هذه التحليلات، يحسب ScoreCalculator الدرجة النهائية باستخدام أوزان ثابتة، ثم يقوم Report Agent بصياغة التقرير النهائي.

القاعدة الأساسية
الذكاء الاصطناعي مسؤول عن التحليل المنظم، أما Final Score فتُحسب داخل Python بشكل Deterministic. بهذا لا يملك LLM صلاحية تغيير النتيجة النهائية.

الأهداف العملية

- تحويل تقييم أفكار Startup من رأي عام إلى عملية تحليل منظمة.
- فصل تحليل الأعمال عن تحليل السوق وعن تحليل المخاطر.
- إظهار Score Breakdown واضح بدلاً من رقم غير قابل للتفسير.
- حساب Final Score بمعادلة ثابتة داخل النظام.
- حفظ جميع التقارير السابقة لكل مستخدم.
- عرض إحصائيات المستخدم وتقاريره الأخيرة داخل Dashboard.

2. الفكرة المعتمدة ونطاق النسخة الحالية

النسخة الحالية هي تطبيق ويب متكامل يبدأ من Authentication وينتهي بتقرير Startup محفوظ داخل PostgreSQL. تركز النسخة الحالية على تحليل الفكرة وإدارة التقارير، بينما بعض التحسينات المتقدمة مؤجلة لمرحلة لاحقة.

البند	الحالة في النسخة الحالية
تسجيل المستخدم وتسجيل الدخول	معتمد
JWT Access و Refresh Token	معتمد
Business Agent	معتمد
Market Agent	معتمد
Risk Agent	معتمد
Report Agent	معتمد
Parallel بتشغيل Agents بشكل	معتمد

ScoreCalculator	Python ثابت داخل	معتمد
PostgreSQL	حفظ التقارير في	معتمد
Dashboard	وتقارير المستخدم	معتمد
Deployment Production		مؤجل للمرحلة التالية
Queue	خارجي Async Worker أو	مؤجل عند الحاجة للتوسع

قرار مهم

الـ Report Agent لا يعيد حساب الدرجة. يستقبل final_score جاهزة من ScoreCalculator ويستخدمها فقط لكتابة التقرير النهائي.

المستخدم

- ينشئ حساباً جديداً أو يسجل الدخول.
- يدخل عنوان فكرة الـStartup ووصفها.
- يبدأ عملية التحليل من صفحة New Analysis.
- يشاهد التقرير النهائي والدرجة والتوصية.
- يرجع إلى جميع تقاريره السابقة.
- يشاهد إحصائياته داخل Dashboard.

الوكلاء الذكيون

المسؤولية	Agent
تحليل المشكلة، الجمهور المستهدف، القيمة المقترحة، الجدوى وقابلية التوسع.	Business Agent
تحليل حاجة السوق، الطلب، الفرصة، المنافسة وإمكانية الوصول.	Market Agent
تحليل مخاطر نموذج العمل، التبني، التنفيذ، التقنية والقانون والخصوصية.	Risk Agent
تجميع النتائج وصياغة التقرير والتوصية النهائية.	Report Agent

طبقات النظام

- Django API: استقبال الطلبات، التحقق من المستخدم وإرجاع JSON Responses.
- ValidationService: تنفيذ Use Case الكامل لعملية Validate Startup.
- StartupAnalysisOrchestrator: تنسيق تشغيل الـAgents.
- ScoreCalculator: حساب الدرجة والتصنيف النهائي.
- HuggingFaceService: طبقة الاتصال بالمودل.
- PostgreSQL: حفظ المستخدمين والأفكار والتقارير والتوكنات المعلقة.

4. طريقة عمل الـMulti-Agent System

الوكلاء الثلاثة الأساسيون Business وMarket وRisk مستقلون ولا يعتمد أحدهم على نتيجة الآخر. جميعهم يستقبلون نفس فكرة الـStartup، لذلك يتم تشغيلهم بالتوازي باستخدام ThreadPoolExecutor مع ثلاثة Workers.

تسلسل العمل

1. يرسل المستخدم title و idea إلى API التحليل.
2. يتحقق jwt_required من Access Token ويحدد request.user.
3. ينشئ ValidationService سجل StartupIdea.
4. يطلق Orchestrator الوكلاء الثلاثة بالتوازي.
5. ينتظر النظام نتائج Business و Market و Risk جميعاً.
6. يحسب ScoreCalculator الدرجة النهائية.
7. يحدد النظام Final Verdict.
8. يستقبل Report Agent التحليلات والدرجة ويكتب التقرير النهائي.
9. يتم حفظ ValidationReport في PostgreSQL.
10. تعود النتيجة إلى الواجهة وتظهر في Report Details.

قرار معماري

Parallel Execution يقلل انتظار الوكلاء المستقلين. أما Report Agent فيبقى بعدهم لأنه يعتمد على نتائج التحليلات الثلاثة والFinal Score.

الميزة	الحالة	ملاحظات
Business تحليل	معتمدة	من Business Score خمسة معايير و 10.
Market تحليل	معتمدة	خمس معايير و Market Score من 10.
Risk تحليل	معتمدة	الدرجة الأعلى تعني خطراً أعلى.
Parallel Agents	معتمدة	ThreadPoolExecutor 3 مع Workers.
Final Score	معتمدة	تُحسب داخل Python.
Final Report	معتمدة	Report Agent لا يغير الدرجة.
حفظ التقارير	معتمدة	PostgreSQL داخل JSONField.
JWT Authentication	معتمدة	Access + Refresh + Revocation.
Dashboard	معتمدة	إحصائيات وتقارير حديثة للمستخدم.
Deployment	مؤجل	Production Settings بعد تجهيز.
CI/CD	مؤجل	في مرحلة لاحقة GitHub Actions.
Queue خارجي	مؤجل	يضاف عند زيادة الحمل ووقت التحليل.

6. التقنيات المستخدمة

المجال	القرار التقني
Backend	Django.
Database	PostgreSQL.
LLM	Qwen/Qwen3-8B عبر Hugging Face InferenceClient.
AI Architecture	Multi-Agent Architecture.
Parallel Execution	concurrent.futures.ThreadPoolExecutor.
Authentication	JWT باستخدام PyJWT.
Environment	python-dotenv وملف .env.
Frontend	Django APIمتصلة بـ Dark Mode واجهة ويب
Version Control	Git و GitHub على branch main.

قرار تقني

تم إنشاء HuggingFaceService كطبقة مستقلة، بحيث لا يعرف كل Agent تفاصيل الـ Provider أو الـ Token. هذا يسمح بتغيير المودل أو مزود الخدمة لاحقاً دون إعادة بناء كل الوكلاء.

مكونات النظام

- Frontend: Login, Dashboard, New Analysis, Reports, Report Details
- Dashboard و Reports و Validation و Django API: Authentication
- AI Agents Layer: Business, Market, Risk, Report
- Workflow و إدارة الـ Agents: Orchestrator تشغيل الـ
- Final Verdict و Final Score حساب: Scoring Service
- Hugging Face عبر Qwen الاتصال: LLM Service
- PostgreSQL: تخزين البيانات المنظمة.

الجدول / Model	الغرض
User	حسابات المستخدمين وإدارة تسجيل الدخول.
StartupIdea	عنوان الفكرة ووصفها وصاحب الفكرة ووقت الإنشاء.
ValidationReport	تحليلات الـ Agents والتقرير والدرجة والتصنيف.
RevokedRefreshToken	حفظ jti الخاص بالـ Refresh Tokens الملغاة.

8. مسارات الاستخدام

مسار المستخدم

- 1. يفتح التطبيق ويسجل الدخول.
- 2. يحصل على Access Token و Refresh Token.
- 3. يفتح صفحة New Analysis.
- 4. يدخل عنوان الفكرة ووصفها.
- 5. يرسل الطلب إلى /api/validate-startup/.
- 6. ينتظر اكتمال التحليل.
- 7. يشاهد Final Score والتقرير النهائي.
- 8. يعود إلى Dashboard لرؤية الإحصائيات.
- 9. يفتح Reports لمشاهدة التحليلات السابقة.

مسار النظام

- 1. التحقق من JWT.
- 2. إنشاء StartupIdea للمستخدم.
- 3. تشغيل Market و Business و Risk Agents بالتوازي.
- 4. انتظار النتائج الثلاثة.
- 5. حساب Final Score.
- 6. توليد Final Report.
- 7. حفظ ValidationReport.
- 8. إرجاع JSON Response إلى الواجهة.

9. نظام التقييم والدرجات

تم اعتماد Score Breakdown داخل كل Agent. يحتوي كل تحليل على خمسة معايير، وكل معيار يأخذ درجة من 0 إلى 2، وبالتالي تكون درجة كل Agent من 0 إلى 10.

المعايير الرئيسية	Agent
Problem Importance, Audience Clarity, Value Proposition, Viability, Scalability	Business
Market Need, Customer Demand, Opportunity, Competitive Position, Accessibility	Market
Business Model Risk, Adoption Risk, Execution Risk, Technology Risk, Legal/Privacy Risk	Risk

معادلة النتيجة النهائية: **Final Score = Business × 0.40 + Market × 0.35 + (10 - Risk) × 0.25**

الدرجة	Final Verdict
أقل من 4	Weak Potential
4 إلى أقل من 6.5	Moderate Potential
6.5 إلى أقل من 8	Good Potential
8 أو أكثر	Strong Potential

Risk - 10 سبب استخدام
لأن Risk Score الأعلى تعني خطراً أعلى. لذلك يتم تحويلها إلى Risk Potential قبل دمجها مع Business وMarket.

المنجز حالياً

- إنشاء مشروع Django وتطبيق validator.
- ربط PostgreSQL.
- بناء Business Agent.
- بناء Market Agent.
- بناء Risk Agent.
- بناء Report Agent.
- بناء StartupAnalysisOrchestrator.
- تشغيل الوكلاء الثلاثة الأساسيين بشكل Parallel.
- بناء ScoreCalculator.
- بناء ValidationService مع transaction.atomic.
- بناء Login و Register و Refresh و Logout.
- إضافة Refresh Token Revocation.
- ربط البيانات بالمستخدم وعزل تقاريره.
- بناء APIs للتحليل والتقارير والـ Dashboard.
- بناء واجهة المستخدم وتجربة سيناريو كامل.
- رفع المشروع إلى GitHub على branch main.

الخطوات التالية

1. تجهيز Production Settings ونقل SECRET_KEY إلى Environment Variable.
2. إضافة Dockerfile و Production Server.
3. تنفيذ Deployment.
4. تحويل Scripts الاختبار إلى Automated pytest Tests.
5. إضافة CI/CD عبر GitHub Actions.
6. دراسة Queue أو Async Processing عند زيادة عدد المستخدمين.

حالة المشروع

الـ Core System يعمل حالياً من Login حتى Dashboard و New Analysis و Report Details، مع حفظ التقارير وتشغيل الـ Agents بالتوازي.

المخاطر	الإجراء المعتمد
إرجاع LLM نتيجة غير منظمة	json.loads فقط ثم JSON صارمة تطلب Prompts
تحكم LLM بالدرجة النهائية	Report وليس داخل ScoreCalculator الحساب داخل Agent.
درجات عامة أو متقاربة	خمس معايير مستقلة لكل Agent ومنع Default Scoring.
رؤية بيانات مستخدم آخر	request.user حسب Dashboard و Reports فلترة
Logout بعد Refresh Token استخدام	RevokedRefreshToken في jti حفظ
زيادة زمن التحليل	بالتوازي Risk و Market و Business تشغيل
أخطاء Provider المؤقتة	xx لأخطاء 5 Retry Logic
تسريب الأسرار	.gitignore. عبر Git مستبعد من .env. ملف

الخلاصة النهائية.

المشروع المعتمد هو نظام ويب لتقييم أفكار الشركات الناشئة باستخدام Multi-Agent Architecture. يبدأ المستخدم بإدخال فكرة Startup، ثم يتم تشغيل Business Agent و Market Agent و Risk Agent بالتوازي. بعد اكتمال التحليلات، يحسب ScoreCalculator الدرجة النهائية بمعادلة ثابتة داخل Python، ثم يستخدم Report Agent النتائج لصياغة تقرير منظم وتوصية واضحة.

يحفظ النظام الأفكار والتقارير داخل PostgreSQL، ويستخدم JWT Authentication لحماية APIs وعزل بيانات كل مستخدم. كما يوفر Dashboard وتقارير سابقة وتجربة استخدام كاملة من Login إلى New Analysis ثم Report Details.

الخلاصة المختصرة

Startup Validator AI ليس مجرد واجهة لنموذج ذكاء اصطناعي. هو نظام برمجي متكامل يجمع Multi-Agent Analysis، REST APIs و Parallel Execution، Deterministic Scoring، PostgreSQL، JWT Authentication ضمن Workflow واحد قابل للتطوير.

شرح المشروع خلال دقيقة

«مشروعي اسمه Startup Validator AI. فكرته أنه يساعد صاحب فكرة Startup يعمل Validation أولي قبل البناء. بدلاً من AI Agent واحد، عندي Multi-Agent System: Business Agent يحلل الجانب التجاري، Market Agent يحلل السوق، و Risk Agent يحلل المخاطر، والثلاثة يعملون بالتوازي. بعدها Final Score لا يحسبها AI، بل ScoreCalculator داخل Python بمعادلة ثابتة، ثم Report Agent يصيغ التقرير النهائي. النظام مبني بـ Django و PostgreSQL ومعه JWT Authentication، لذلك كل مستخدم يرى أفكاره وتقاريره فقط.»

الجملة الأساسية لشرح قوة المشروع

AI يحلل، الOrchestrator ينسق، Python يحسب، PostgreSQL يحفظ، وJWT يحمي البيانات.